

# Correção do campo role no PostgreSQL e Spring Boot

Este PDF reúne os comandos usados para corrigir o problema do enum UserRole quando o banco salva valores numéricos como 0/1, mas a aplicação Java espera ADMIN/USER.

## 1. Problema encontrado

O erro apareceu durante o login/autenticação:

```
org.springframework.security.authentication.InternalAuthenticationServiceException:  
No enum constant com.nitssrpi.NIT_SRPI.model.UserRole.1
```

Isso acontece porque o PostgreSQL estava com o campo role como smallint, salvando 0 ou 1. Porém, com @Enumerated(EnumType.STRING), o Spring/JPA espera encontrar no banco os nomes do enum: ADMIN ou USER.

## 2. Enum Java utilizado

```
public enum UserRole {  
    ADMIN("admin"),  
    USER("user");  
  
    private String role;  
  
    UserRole(String role){  
        this.role = role;  
    }  
  
    public String getRole(){  
        return role;  
    }  
}
```

Importante: mesmo existindo "admin" e "user" dentro do enum, com EnumType.STRING o valor salvo no banco deve ser o nome do enum: ADMIN ou USER.

## 3. Ajuste necessário na entidade User

```
@Enumerated(EnumType.STRING)  
@Column(nullable = false, length = 20)  
private UserRole role;
```

## 4. Comandos SQL corretos para corrigir o banco

Rode os comandos abaixo nessa ordem no PostgreSQL.

### 4.1 Verificar o tipo atual da coluna role

```
SELECT column_name, data_type  
FROM information_schema.columns  
WHERE table_schema = 'public'  
AND table_name = 'users'  
AND column_name = 'role';
```

### 4.2 Remover qualquer CHECK antigo envolvendo role

```
DO $$  
DECLARE  
    constraint_record RECORD;
```

```

BEGIN
    FOR constraint_record IN
        SELECT conname
        FROM pg_constraint c
        JOIN pg_class t ON c.conrelid = t.oid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        WHERE t.relname = 'users'
        AND n.nspname = 'public'
        AND c.contype = 'c'
        AND pg_get_constraintdef(c.oid) ILIKE '%role%'
    LOOP
        EXECUTE format(
            'ALTER TABLE public.users DROP CONSTRAINT IF EXISTS %I',
            constraint_record.conname
        );
    END LOOP;
END $$;

```

### 4.3 Converter a coluna role para texto e ajustar os valores

```

ALTER TABLE public.users
ALTER COLUMN role TYPE varchar(20)
USING CASE
    WHEN role::text = '0' THEN 'ADMIN'
    WHEN role::text = '1' THEN 'USER'
    WHEN lower(role::text) = 'admin' THEN 'ADMIN'
    WHEN lower(role::text) = 'user' THEN 'USER'
    WHEN role::text = 'ADMIN' THEN 'ADMIN'
    WHEN role::text = 'USER' THEN 'USER'
    ELSE 'USER'
END;

```

### 4.4 Garantir que não ficou valor inválido

```

UPDATE public.users
SET role = CASE
    WHEN role = 'ADMIN' THEN 'ADMIN'
    WHEN role = 'USER' THEN 'USER'
    ELSE 'USER'
END;

```

### 4.5 Criar o role\_check correto

```

ALTER TABLE public.users
ADD CONSTRAINT users_role_check
CHECK (role IN ('ADMIN', 'USER'));

```

### 4.6 Conferir o resultado final

```

SELECT id, email, username, role
FROM public.users;

```

O resultado esperado é que a coluna role tenha apenas os valores ADMIN ou USER.

## 5. Observação sobre login por e-mail

Como sua API de login recebe email e password, o UserDetailsService deve buscar usuário por e-mail. Além disso, se o login for por e-mail, o getUsername() da entidade User pode retornar o email.

```

@Override
public String getUsername() {
    return this.email;
}

```

## 6. Resumo simples

| Antes                              | Depois                                        |
|------------------------------------|-----------------------------------------------|
| role smallint                      | role varchar(20)                              |
| 0 / 1                              | ADMIN / USER                                  |
| Enum salvo como número             | Enum salvo como texto                         |
| Erro no login ao carregar UserRole | Spring consegue montar o usuário corretamente |

Depois da correção, o banco passa a salvar o mesmo formato que o JPA espera quando a entidade usa `@Enumerated(EnumType.STRING)`.